STUDENT GUIDE / ENGLISH

DryRun Security

Learn by doing, step by step

From first-time setup to AI-assisted repair

No previous Git or terminal experience required

---

01 **PREPARE**

Tools, accounts and a safe exercise repository

02 **EXECUTE**

25 modules with 167 guided steps

03 **VERIFY**

Expected outcomes and observable evidence

04 **RECOVER**

Troubleshooting, resuming and cleanup

GITHUB / DRYRUN API / CODEX & MCP / JIRA

Edition 2 • 2026-10-23 • Independent training material

Contents

How to study this course 3
Reading the blocks 4
Initial setup - do once 5
Resume on another day 7
Additional setup - when you reach integrations 8
01. Documentation 9
02. Quick Start 11
03. PR Scanning 14
04. SCA - Dependencies 17
05. Repository Scanning with DeepScan 19
06. Secrets Scanning 21
07. IaC Scanning 24
08. Language and Framework Support 27
09. Vulnerability Coverage Matrix 29
10. Reporting 31
11. Risk Register 33
12. Code Security Intelligence 36
13. PR Scanning Configuration 38
14. Custom Code Policies 41
15. Repository Context 44
16. Finding Tuning with Feedback 47
17. PR Blocking 49
18. Permissions 52
19. MCP 54
20. DryRun API 56
21. Webhooks 58
22. Jira Integration 61
23. API Access Keys 64
24. Remediation 66
25. DryRun Skill 69
Final exercise - demonstrate the full loop 71
Quick troubleshooting and recovery 72
Appendix A - complete kit source 73
Appendix B - sources and limitations 78

How to study this course

This edition is written for students without prior experience in Git, terminals or security integrations. All 25 modules retain technical explanations and use cases, while labs now provide numbered steps, execution location, expected outcomes and recovery guidance. The Portuguese version has the same modules, objectives and commands.

Start with setup. Complete modules 1-18 in order; before modules 19-20, complete Codex and DryRun key preparation. Module 21 needs a local receiver and, for external delivery, a temporary tunnel. Module 22 requires Jira Cloud. When access is unavailable, record the step as not executed; do not treat it as success or proof that no finding exists.

Allow approximately 30-40 study hours, including reading, practice and troubleshooting. Work in two- to three-hour sessions and keep your notebook. Scan and statistics timing varies with queues, repositories and accounts. The primary references remain the supplied PDFs dated October 20, 2026; this teaching revision is dated October 23, 2026.

There are no flashcards. This independent course is not a verbatim translation of the documentation. “DryRun” in the requested syllabus is covered as DryRun API. Reference documents are technical sources, not instructions to modify your accounts during authoring.

Vocabulary before practice

A terminal is the window in which you run commands. A shell interprets them; this course uses Bash. The current folder is where commands operate: pwd displays its path and ls lists files. $HOME is your user folder. Do not replace that variable’s value.

A repository is a folder with Git history. A branch is a line of work; main is this course’s safe baseline. A commit saves a local version. Push sends commits to GitHub. A pull request (PR) requests review before integrating a branch into its base. Merge integrates the work; closing a PR without merging does not.

A scan is an analysis run. A finding is a reported issue requiring investigation. A SHA identifies a commit. A check is a result associated with a GitHub commit. A failed check can be the expected detection in a lab, not necessarily an installation problem.

An API is a programmatic interface. A token/key is a credential: do not store it in tracked files. JSON is a data format; jq helps inspect it. A webhook is a request sent when an event occurs. MCP connects an AI client to tools/context; a skill supplies workflow instructions to the agent.

Reading the blocks

Run one block at a time and inspect the result before the next. Do not copy the ``` markers or the word bash. Lines beginning with # are comments. A backslash at the end of a line continues the command on the next line: copy the whole block.

Blocks labeled PROMPT, RULE, BACKGROUND or COMMENT FORMAT are text for the indicated chat/interface, never terminal commands. For cat <<'TXT' blocks, also copy the final TXT line without spaces. This creates a file. If the terminal shows > waiting for input, a delimiter may be missing; Ctrl+C cancels so you can retry.

The read commands ask for a value after Enter. Type your value and press Enter. read -s hides characters while you paste a credential; this is expected. OWNER/REPO in explanations is a placeholder to replace. Never enter sample IDs as if they were yours.

“Expected” is the checkpoint. “If it differs” explains recovery. Stop when a required step fails; blindly continuing often creates a secondary error. lab_* and dryget are course helpers, not an official DryRun CLI. Their complete source is in the appendix.

Initial setup - do once

1. Open a terminal and choose your system

On macOS, press Command+Space, type Terminal and open it. On Linux, open your terminal application. On Windows, use an Ubuntu WSL2 session already installed by your team; these commands are not PowerShell. The main path uses Homebrew on compatible macOS or Linux/WSL. If your company manages installations, ask IT to supply the tools rather than bypassing policy.

Type the following and press Enter. This window now uses Bash even if your macOS default is zsh.

```
bash
```

2. Install the basic tools

First check whether Homebrew exists. If a path appears, skip installation and proceed to brew install.

```
command -v brew
```

If nothing appears, use the official installer below. Read its explanation and follow its prompts. It may request your system password and developer tools. Resolve installation errors before continuing. Reference: https://brew.sh

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

The installer displays “Next steps”. Follow those instructions. To enable the path in this session, one of these standard locations should exist; this block selects the matching system location.

```
if [ -x /opt/homebrew/bin/brew ]; then
  eval "$(/opt/homebrew/bin/brew shellenv)"
elif [ -x /usr/local/bin/brew ]; then
  eval "$(/usr/local/bin/brew shellenv)"
elif [ -x /home/linuxbrew/.linuxbrew/bin/brew ]; then
  eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"
fi
brew install git gh jq python node
```

If Homebrew requests prerequisites on Ubuntu Linux/WSL, install them and resume the installer. Do not run this block on macOS.

```
sudo apt-get update
sudo apt-get install -y build-essential procps curl file git openssh-client
```

Verify the tools. Each command should print a version or path. command not found means installation/PATH needs correction. curl must support --fail-with-body; if the system curl is old, install a recent version through the package manager and confirm that version is on PATH.

```
git --version
gh --version
jq --version
python3 --version
node --version
npm --version
curl --version
command -v ssh-keygen
```

3. Sign in to GitHub

Run login. In the wizard choose GitHub.com, HTTPS and browser login. Copy the temporary code, open the indicated page and authorize your own account. Labels may change; the goal is to authenticate the CLI with the account that can access the exercise repository.

```
gh auth login
gh auth setup-git
gh auth status
```

Expected: your account is authenticated on github.com. If another account appears, correct it before creating a repo. setup-git configures Git authentication through the CLI; it does not create repositories.

4. Download and inspect the materials

This clone contains materials, not the scan target. Module 2 creates a different repository for exercises. Do not create vulnerable exercise branches inside material.

```
mkdir -p "$HOME/dryrun-student"
git clone https://github.com/lca-appsec/DryRun-training.git \
  "$HOME/dryrun-student/material"
ls "$HOME/dryrun-student/material/laboratorios"
mkdir -p "$HOME/dryrun-student/notes"
```

You should see app.py, test_app.py, lab.sh and save_session.py. If the folder exists, do not clone again: check the repository and update materials only when there are no local changes.

```
git -C "$HOME/dryrun-student/material" status --short
git -C "$HOME/dryrun-student/material" pull --ff-only
```

Offline alternative: extract the kit ZIP into $HOME/dryrun-student/material/laboratorios. That materials folder will not be a Git clone, so do not run pull there. Creating the exercise repository still requires network access.

5. Verify access

Open https://app.dryrun.security and confirm your organization. The App will be enabled for the new repo in module 2. Confirm the administrative role needed for configuration, policies, DeepScan and keys. Jira and Codex require their own access; GitHub authentication does not grant it automatically.

Resume on another day

After module 2, session.sh stores paths and OWNER/REPO without keys. In any new terminal run the commands below. Do not recreate the repo or overwrite app.py with a fresh copy.

```
bash
source "$HOME/dryrun-student/session.sh"
pwd
git branch --show-current
git status --short
```

Expected: pwd ends in dryrun-student/repo. If you stopped on lab/03, resume that module's step. lab_branch creates a new branch and should not be repeated if it already exists. To inspect an existing PR, use gh pr view --web on its branch.

If a lab_* command is unavailable, load session.sh. If session.sh is missing but the repo exists, enter repo, inspect its remote and run save_session.py with the actual OWNER/REPO. This command prompts for the name instead of assuming a value.

```
cd "$HOME/dryrun-student/repo"
git remote -v
read -rp 'Existing exercise OWNER/REPO: ' GH_REPO
python3 "$HOME/dryrun-student/material/laboratorios/save_session.py" "$GH_REPO"
source "$HOME/dryrun-student/session.sh"
```

For src refspec main does not match any, run git status, git branch --show-current and git log -1. Without an initial commit, finish module 2. If the branch has another name, understand its history before renaming. Do not use force push or reset --hard to repair setup. Do not copy the HTML text   into commands.

Additional setup - when you reach integrations

DryRun key - before modules 19 and 20

In DryRun, open Settings > Access Keys, create a key and name it training-read. Select at least one scope; choose the read scopes needed for the exercise data. Inspect actual scopes in your account instead of inferring names from endpoints. Save, copy the once-displayed secret and store it in a secure manager.

In the terminal where you will call the API or launch Codex, paste the token when prompted. Input is hidden. Press Enter to finish.

```
source "$HOME/dryrun-student/session.sh"
read -rsp 'DryRun API key: ' DRYRUN_API_KEY; echo
export DRYRUN_API_KEY
dryget accounts
```

Expected: JSON listing authorized accounts. For 401 check the token; for 403 check scope/role. Do not put the token in session.sh, README or Git. Read/export it again in a new terminal. Module 23 covers rotation and revocation.

Codex - before modules 19, 24 and 25

Install and authenticate the CLI in your terminal. Follow the opened login page and return to check status. Official references: https://learn.chatgpt.com/docs/cli/reference and https://learn.chatgpt.com/docs/codex/cli

```
npm install -g @openai/codex
codex --version
codex login
codex login status
```

If npm reports permission denied, use a user-managed Node installation or ask IT; do not blindly add sudo. Login requires an account authorized to use Codex. Module 19 configures MCP separately.

cloudflared - before module 21 external delivery

Install through the same Homebrew used in setup. This client creates a temporary HTTPS URL for a local receiver; no production service needs to be opened.

```
brew install cloudflared
cloudflared --version
```

Jira Cloud: obtain a site, test project, creation permission and personal token before module 22. Query, payload and creation commands are included in that module. Do not use a production project as a test inbox.

GETTING STARTED

01. Documentation

Technical explanation

DryRun relates code changes to surrounding functions, data flows and security controls. The analysis unit matters: PR Scanning reviews proposed changes, DeepScan explores a repository reference, SCA examines dependencies, and Risk Register organizes findings. A clean PR scan is not a full application audit.

A source-control event starts a scan. Results can appear in checks, comments, the dashboard and enabled integrations. A status check reports a result; a GitHub protection rule makes that check a merge requirement. These are separate configuration decisions.

LLMs support contextual analysis and Code Security Intelligence. MCP connects an AI client to tools and authorized context; it is not itself a scanner. Ask an assistant to identify the evidence behind its conclusions, such as paths, lines, commits and finding IDs.

Use case

A team reviews an authorization change. It checks the PR diff, documents the actual authorization design with Repository Context, investigates legacy variants with DeepScan, and records a reasoned treatment decision.

Before you start

Initial setup completed; browser access to GitHub and DryRun. The exercise repository is not needed yet.

Guided lab 01

Step 1

Terminal - check the authenticated GitHub account. This reads your identity without changing repositories.

```
gh auth status
gh api user --jq .login
```

Expected: Write down the returned login. You will use it in module 2.

If it differs: If authentication fails, repeat gh auth login from the setup section.

Step 2

Browser - open https://app.dryrun.security, sign in and select the correct organization. Locate Repository Scanning/DeepScan, Risk Register and Settings; labels may vary.

Expected: You can identify where to view scans, handle findings and configure the product.

If it differs: If the organization is missing, ask its administrator for access; do not use a different organization for convenience.

Step 3

Terminal - create a local notebook. Fill the blank lines in a plain-text editor. This file will not be uploaded to GitHub.

```
mkdir -p "$HOME/dryrun-student/notes"
cat > "$HOME/dryrun-student/notes/01.txt" <<'TXT'
GitHub login:
DryRun organization:
My role:
PR scan = changed code
DeepScan = repository analysis
TXT
```

Expected: Save the identity and role shown by the platform.

If it differs: If you do not know your role, record “unconfirmed” and confirm it before changing settings.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Documentation, pages 1-2; Quick Start, pages 1-7.

GETTING STARTED

02. Quick Start

Technical explanation

The GitHub App must be installed and allowed to access the exercise repository. Organization membership alone does not guarantee that a newly created repository is selected in an installation restricted to specific repositories. Complete onboarding by observing a real scan, not merely by seeing the App installed.

The course starts with a safe baseline and two tests: a normal lookup and an injection string treated as a literal value. Every exercise branch starts from main. Vulnerable exercise PRs are closed without merging, so the next module does not inherit the previous fault.

Git stores commits locally before push uploads them. The error src refspec main does not match any usually means there is no local main reference, often because there is no first commit or the branch has another name. A remote is a destination, not a branch.

Use case

A dedicated private repository lets a student experiment with findings and settings without changing a production project or the training-material repository.

Before you start

Module 1 and setup completed. Choose an account/organization where you can create a private repo and authorize the App.

Guided lab 02

Step 1

Terminal - run the commands and enter your account/organization when prompted. Choose a new repo name, such as dryrun-student-luis. Do not use DryRun-training, which stores course materials.

```
read -rp 'GitHub owner: ' GH_OWNER
read -rp 'New exercise repository name: ' LAB_NAME
export GH_REPO="$GH_OWNER/$LAB_NAME"
mkdir -p "$HOME/dryrun-student/repo"
cd "$HOME/dryrun-student/repo"
git init -b main
read -rp 'Your commit name: ' COMMIT_NAME
read -rp 'Your commit email: ' COMMIT_EMAIL
git config --local user.name "$COMMIT_NAME"
git config --local user.email "$COMMIT_EMAIL"
```

Expected: git init creates .git and identifies main. Name/email identify commits; they are not passwords.

If it differs: If this folder already contains your course repo, do not reinitialize or overwrite it; use Resume.

Step 2

Terminal - copy only the application and tests from the kit. Run the test before creating the first version.

```
export KIT="$HOME/dryrun-student/material/laboratorios"
cp "$KIT/app.py" "$KIT/test_app.py" .
printf '.evidence/\n__pycache__/\n.env\n.venv/\n' > .gitignore
printf '# DryRun student repository\n' > README.md
python3 -m unittest -v
```

Expected: You should see two tests and OK. No server installation or external database is required.

If it differs: If app is not found, run pwd and ls; app.py and test_app.py must be in this folder.

Step 3

Terminal - save the first commit and push it to a new private repo. This publishes the safe source baseline, not an Internet-facing application.

```
git add app.py test_app.py .gitignore README.md
git commit -m 'Safe student baseline'
gh repo create "$GH_REPO" --private --source . --remote origin --push
python3 "$KIT/save_session.py" "$GH_REPO"
source "$HOME/dryrun-student/session.sh"
```

Expected: GitHub returns the repo URL. session.sh stores paths and the repo name, without tokens.

If it differs: If the name exists, do not force push. Confirm it is your exercise repo and resume it, or choose another name.

Step 4

Browser - in the DryRun GitHub App installation, choose Configure and include the new repo under Repository access. Confirm it appears in DryRun. If you are not an administrator, ask one to perform this step.

```
gh repo view "$GH_REPO" --web
```

Expected: The same OWNER/REPO name appears in GitHub and DryRun.

If it differs: Do not proceed to scanning if the installation still excludes the repo.

Step 5

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/02
```

Expected: The last message should identify branch lab/02. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 6

Terminal - make a documentation change and save a commit.

```
printf '\nFirst onboarding PR.\n' >> README.md
git add README.md
git commit -m 'Test DryRun onboarding'
```

Expected: The commit contains only README.md.

If it differs: Use git diff --stat and git status to inspect unexpected files before publishing.

Step 7

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 02 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Quick Start, pages 1-7.

SCANNING

03. PR Scanning

Technical explanation

PR Scanning evaluates the diff and the context needed to understand its security impact. New commits trigger another analysis. Legacy code outside the change is not thereby fully audited. Draft PRs may be scanned while comments wait until the PR is ready for review.

By default a clean scan may produce no comment, and a previously posted comment may disappear after all findings are fixed. Show comment for no findings changes this behavior. Always check the scan/check result and commit rather than using comment presence as the sole signal.

SQL injection occurs when untrusted input changes SQL structure. This exercise replaces parameter binding with string concatenation. A local regression test demonstrates the defect independently of whether the scanner reports it. The repair restores binding while preserving the normal query result.

Use case

A developer simplifies a search function by concatenating the HTTP query parameter into SQL. The review traces the request input through search to sqlite3.execute and verifies the proposed repair.

Before you start

Module 2 completed; safe main; repo visible in DryRun.

Guided lab 03

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/03
```

Expected: The last message should identify branch lab/03. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - introduce the fault only on this branch and run the tests. unsafe selects the deliberately vulnerable version.

```
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git diff -- app.py
```

Expected: test_injection_is_literal should fail; the normal lookup test passes. This failure is expected and proves the defect.

If it differs: If both pass, stop: check whether the change was applied and whether you are in the correct repo folder.

Step 3

Terminal - commit the vulnerable version for review.

```
git add app.py
git commit -m "Lab: unsafe SQL query"
```

Expected: The commit diff shows SQL concatenation.

If it differs: If nothing to commit appears, inspect git log -1; you may already have done this step.

Step 4

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 03 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 5

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 6

DryRun - open the finding if present. In your notebook, record ID, file, line, controlled input, SQL operation and severity. Compare the explanation to app.py.

```
nl -ba app.py
```

Expected: You can explain why concatenation allows changing query structure.

If it differs: A scan without a finding does not invalidate the local proof. Record both results.

Step 7

Terminal - fix, test and push a new commit to the same PR.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Fix SQL query using bound parameters'
git push
```

Expected: Both tests pass. The same PR receives a new SHA and scan.

If it differs: Do not close the PR before checking the corrected commit's result.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: PR Scanning, pages 1-5; PR Scanning Configuration, pages 3-4.

SCANNING

04. SCA - Dependencies

Technical explanation

Software Composition Analysis identifies components and known vulnerabilities from supported manifests and lockfiles. A dependency range is not necessarily the installed version; a lockfile records resolution and improves inventory accuracy. A software bill of materials (SBOM) is an inventory, not an equivalent list of vulnerabilities.

DryRun documents SCA within DeepScan and background scans scheduled according to PR activity over the previous 30 days. Repositories with fewer than three PRs are not included in that background schedule. Use an explicit DeepScan for this new training repository instead of waiting for background activity.

For Gradle, build.gradle alone is insufficient: use a supported committed lockfile. The documentation also describes root-level verification-metadata.xml support. Review direct and transitive dependencies, actual versions and compatibility before choosing a remediation.

Use case

A team records an inventory before upgrading a dependency. It uses the finding and advisory to choose a supported replacement, updates the project’s dependency resolution and tests compatibility.

Before you start

Module 2; permission to start DeepScan. You do not need to install requests.

Guided lab 04

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/04
```

Expected: The last message should identify branch lab/04. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - create a manifest with an old version for analysis. This writes a file; it does not install the library.

```
printf 'requests==2.19.1\n' > requirements.txt
git add requirements.txt
git commit -m 'Lab: dependency manifest'
```

Expected: requirements.txt contains exactly requests==2.19.1.

If it differs: Do not run pip install for this exercise. The goal is to analyze the committed manifest.

Step 3

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 04 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 4

DryRun - open Repository Scanning/DeepScan, select your repo and choose branch lab/04 in the reference selector. Start the scan. Record the reference before confirming.

```
git rev-parse HEAD
```

Expected: The run identifies the exercise branch. Wait for completion before inspecting dependencies.

If it differs: If only main can be selected, stop and confirm the branch workflow with the administrator; main does not contain this manifest.

Step 5

DryRun - open the run's SCA/dependency results, find requests and check the version. Export the SBOM using the available control and save it under .evidence in the browser.

Expected: Record the component, version and finding/advisory if present. No particular CVE is guaranteed.

If it differs: If the component is missing, check the branch, committed file, supported format and SCA completion.

Step 6

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: SCA, pages 1-6; DryRun API, pages 1-6.

SCANNING

05. Repository Scanning with DeepScan

Technical explanation

DeepScan assesses the repository beyond the current diff. The documented flow profiles the project, maps routes and authentication, plans investigation by security domain, reviews code, consolidates/calibrates findings and produces a report. It is useful for a baseline or investigation of legacy code.

Git history provides prioritization signals such as change frequency, contributors, recency, complexity and files that change together. These signals guide investigation; they are neither proof of a vulnerability nor a developer performance rating.

Record the analyzed branch and commit. A report from an older reference does not validate current code. DeepScan may discover nested AGENTS.md files, while specifically configured DeepScan context replaces the AGENTS.md context for that analysis.

Use case

An inherited service contains authorization code that has not changed recently. A repository scan investigates that legacy surface, then the team creates targeted repair PRs.

Before you start

Module 2; Admin or appropriate DeepScan permission.

Guided lab 05

Step 1

Terminal - switch to main and record its commit. No PR is needed for this exercise.

```
source "$HOME/dryrun-student/session.sh"
git status --short
git switch main
git rev-parse HEAD
```

Expected: The working tree is clean and you have main’s SHA.

If it differs: If there are changes, finish the previous lab before switching branches.

Step 2

DryRun - open DeepScan, select the repo and main, review the target and start one run. Record the time and displayed identifier.

Expected: A queued or processing run appears. This is not yet a final result.

If it differs: If the control is unavailable, check role and account features; do not retry through the API to bypass permission.

Step 3

DryRun - follow the phases until completion. Open the report and locate languages, routes/surfaces and findings where provided. Compare it to app.py.

```
nl -ba app.py
```

Expected: The report matches the selected target. A small safe baseline may have zero findings.

If it differs: If it fails, save the error and ID. Do not start duplicate runs to speed up the queue.

Step 4

Notebook - save repo, branch, commit, ID, final state and one coverage observation. Stay on main; there is no PR to close.

```
git status --short
```

Expected: You can associate a report with an exact code version.

If it differs: If the report does not expose the commit, record that limitation instead of asserting SHA equality.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Repository Scanning with DeepScan, pages 1-5.

SCANNING

06. Secrets Scanning

Technical explanation

Secrets scanning examines potential credentials in source code and uses context to distinguish suspicious material from examples. The documentation describes attempts to validate activity where applicable. A detection still needs operational assessment; deleting a real leaked credential from the current file does not revoke it.

Dummy values may not be reported because their format or context is not credible. This lab generates a disposable private key that is never trusted by any server or account. Detection is not guaranteed; recording non-detection correctly is also part of the exercise.

For a real exposed secret, revoke/rotate it at its provider, update consumers, verify operation and then handle source/history according to your incident process. There is no universal DryRun command to revoke every kind of external credential.

Use case

A developer commits a key. The team distinguishes the private key from its public counterpart and handles exposure at the provider, not merely by deleting a line in the PR.

Before you start

Module 2; system ssh-keygen available. Never use a personal key.

Guided lab 06

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/06
```

Expected: The last message should identify branch lab/06. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - generate a disposable key without a passphrase. It belongs only to this exercise; do not register its public key with GitHub or any server.

```
mkdir -p fixtures
ssh-keygen -t rsa -b 2048 -N '' \
  -C 'student-never-trusted' -f fixtures/lab_only_key
git add fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Lab: disposable untrusted key'
```

Expected: Two files are created: the private key and public .pub file.

If it differs: If asked to overwrite a file, answer n and check whether this lab was already performed.

Step 3

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 06 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 4

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 5

DryRun - if there is a finding, check the referenced file and classification. Record the ID and conclusion without copying the full key into your notebook.

Expected: You distinguish private from public material and know the key was never trusted.

If it differs: If there is no finding, check agent/settings; do not use a real secret to force detection.

Step 6

Terminal - remove both files in a new commit. Then inspect the updated scan and close the PR.

```
git rm fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Remove disposable key files'
git push
```

Expected: The files leave the current version but remain in branch history.

If it differs: This would be insufficient for a real secret; this exercise key was never used or trusted.

Step 7

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 8

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Secrets Scanning, pages 1-2.

SCANNING

07. IaC Scanning

Technical explanation

Infrastructure-as-code scanning reviews configuration source, including the documented Terraform .tf PR workflow. Relevant risks include broad permissions, network exposure, unsafe defaults and subdomain-takeover-related conditions. You do not need to deploy infrastructure for this exercise.

Context changes meaning: public access to an administrative port is different from an intentionally public service endpoint. An overly broad IAM policy also needs a concrete explanation of resources, actions and intended trust boundaries.

The exercise changes SSH exposure from every IPv4 address to a documentation-only range. That range is not a production access design, and the sample is not a deployable complete Terraform environment.

Use case

A PR opens port 22 to the Internet. A reviewer identifies the exposure and narrows the source to the approved administrative network after checking the actual architecture.

Before you start

Module 2. No AWS account or terraform apply is required.

Guided lab 07

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/07
```

Expected: The last message should identify branch lab/07. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - copy the entire block, including the final TF line. It creates Terraform source without provisioning anything.

```
mkdir -p infra
cat > infra/security.tf <<'TF'
resource "aws_security_group" "lab" {
  name = "dryrun-student"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
TF
git add infra/security.tf
git commit -m 'Lab: broad SSH ingress'
```

Expected: The file contains port 22 and 0.0.0.0/0.

If it differs: If the terminal waits for more text, check that TF was copied without spaces. Ctrl+C cancels input so you can retry.

Step 3

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 07 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module’s branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 4

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 5

Terminal - restrict the example to a documentation-only range. Compare the change and push it.

```
python3 - <<'PYCODE'
from pathlib import Path
p = Path('infra/security.tf')
p.write_text(p.read_text().replace('0.0.0.0/0', '192.0.2.0/24'))
PYCODE
git diff
git add infra/security.tf
git commit -m 'Lab: narrow administrative source'
git push
```

Expected: The diff changes only the range. A real project needs an actual authorized source.

If it differs: Do not run apply; this is source analysis, not deployed-infrastructure testing.

Step 6

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 7

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: IaC Scanning, pages 1-2.

SCANNING

08. Language and Framework Support

Technical explanation

Documented languages include JavaScript/TypeScript, Python, Java, Go, Ruby, PHP, C#, Kotlin, Swift and Rust. Automatic recognition does not imply identical coverage of every framework, generated language or internal library within each ecosystem.

Evaluate coverage by checking that the analysis understands the entry point, the propagation path, the sensitive operation and the relevant controls. Context can explain proprietary wrappers but cannot guarantee analysis of every custom pattern.

Repository language statistics describe composition, not proof of security coverage. GitHub language statistics may use byte counts and generated-file rules that differ from DryRun's analysis.

Use case

A polyglot team samples a representative route in each stack before generalizing coverage. A successful Python finding does not prove that a Java middleware wrapper is understood.

Before you start

Modules 2 and 5. No new PR in this lab.

Guided lab 08

Step 1

Terminal - load your session and list tracked files. .py identifies Python files.

```
source "$HOME/dryrun-student/session.sh"
git switch main
git ls-files
```

Expected: You see app.py and test_app.py, plus documentation/Git configuration.

If it differs: If previous-lab files appear, check that you are on main and did not merge them accidentally.

Step 2

Terminal - query GitHub's language composition. Then open module 5's DeepScan profile in DryRun.

```
gh api "repos/$GH_REPO/languages"
```

Expected: Compare Python recognition across sources without expecting identical numbers.

If it differs: An empty response for a newly created repo may require waiting for GitHub statistics processing.

Step 3

Terminal and notebook - find Handler.do_GET, search and sqlite3.execute. Record input, propagation, operation and control.

```
nl -ba app.py
```

Expected: Map: query q -> search -> execute; control: bound parameter.

If it differs: If the report does not expose a detail, mark it “not observed”; do not fill evidence by assumption.

Step 4

Notebook - conclude that this sample evaluates Python/standard-library code. Another framework needs a separate authorized sample. No PR cleanup is needed.

```
git status --short
```

Expected: The working tree remains clean and the conclusion has explicit scope.

If it differs: Do not claim coverage of every framework from this one exercise.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Language and Framework Support, pages 1-2.

SCANNING

09. Vulnerability Coverage Matrix

Technical explanation

The matrix describes vulnerability categories and associated CWEs. Use it to plan investigation, not as a guarantee that every occurrence of every CWE will be detected. Different classes require different evidence: data flow, authorization, cryptography, configuration or business logic.

Group the categories into injection, authentication/authorization, data exposure, resource access, cryptography/configuration and LLM-related risks. The supplied matrix includes LLM Tool Misuse and Prompt Injection with CWE-1426 and CWE-1427. Evaluate where untrusted content influences a privileged action, rather than looking only at prompt text.

Threat modeling starts with assets, actors, entry points, trust boundaries, abuse paths and intended controls. DryRun can provide code evidence through scans and CSI; a human still determines business impact, validates assumptions and decides treatment.

Use case

An assistant reads untrusted tickets and can call a privileged tool. Investigate whether ticket content can influence tool arguments without independent authorization.

Before you start

Modules 3 and 5; notebook available.

Guided lab 09

Step 1

Browser - open the official matrix linked in this module’s reference. Locate SQL Injection and the LLM tool misuse/prompt injection category. Address: https://docs.dryrun.security/vulnerability-coverage-matrix

Expected: You find the class, description and associated CWE.

If it differs: Do not confuse a CWE, which describes a class, with a finding ID from your run.

Step 2

Terminal - create a hypothesis file under .evidence. It distinguishes a demonstrated fault from systems not implemented in this repo.

```
source "$HOME/dryrun-student/session.sh"
mkdir -p .evidence
cat > .evidence/threats.txt <<'TXT'
TM01 SQLi: q -> search -> SQL. Demonstrated in lab 03.
TM02 IDOR: cross-user invoice access. Not implemented here.
TM03 LLM tool misuse: untrusted ticket -> tool. Not implemented here.
TXT
cat .evidence/threats.txt
```

Expected: The file contains three threats with different evidence states.

If it differs: If .evidence does not exist, mkdir -p creates it; Git ignores it.

Step 3

Notebook - for TM01 record asset (products), input (q), boundary (HTTP/SQL), control (binding) and evidence (test and scan). For TM02/TM03 record “untested hypothesis”.

Expected: You do not create confirmed-vulnerability tickets for hypotheses lacking implementation/evidence.

If it differs: If a scan is empty, do not mark every threat resolved.

Step 4

Notebook - choose another matrix class and describe what code would be needed to test it. Save the file; there is no PR to close.

```
git status --short
```

Expected: The result is a scoped assessment plan, not a security certificate.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Vulnerability Coverage Matrix, pages 1-5.

PLATFORM

10. Reporting

Technical explanation

Reporting presents activity and outcomes over windows such as 24 hours, 7, 30 and 90 days. Metrics include scanned PRs, findings, merged PRs and risk reaching merge. Improvement reflects findings disappearing in subsequent scans; it is not equivalent to a manual audit of every repair.

PR counts, scan counts and finding counts are different units. One PR can have multiple scans and findings. A decrease can reflect reduced activity, different scope, changed configuration or actual remediation. Interpret the number alongside its denominator and time window.

An API export of findings is a useful custom dataset, but it does not automatically reproduce dashboard aggregates. Pagination, latest-run selection, branches and state filters change what is counted.

Use case

A manager evaluates whether risky changes are fixed before merge, considering volume, exceptions and configuration changes rather than ranking teams by raw finding count.

Before you start

Modules 2 and 3 executed; Reporting visibility.

Guided lab 10

Step 1

DryRun - open Reporting and select 7 days. Check organization and available repository filters. Do not change scanning settings.

Expected: The window and scope are visible for recording.

If it differs: If no data appears, try 30 days and verify the organization; do not invent activity to fill the table.

Step 2

Terminal - prepare a report record; then fill it with the numbers and units displayed in the dashboard.

```
source "$HOME/dryrun-student/session.sh"
cat > .evidence/report.txt <<'TXT'
Observed at / time zone:
Window:
Repository scope:
Scanned PRs:
Findings:
Merged PRs with risk:
Configuration changes:
Interpretation and limitations:
TXT
```

Expected: You have a worksheet without treating blank values as zero.

If it differs: Mark unavailable fields “not available”.

Step 3

GitHub - list your PRs and compare one course PR to the dashboard. A PR can have multiple scans.

```
gh pr list --state all --limit 30
```

Expected: You can explain why scan and PR counts need not match.

If it differs: The list is limited to 30; do not present it as complete history if more PRs exist.

Step 4

Notebook - write one interpretation and one limitation. For example, exercises corrected before closing do not represent production merges. Save; no PR needs closing.

Expected: The report includes source, window, unit and limitation.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Reporting, pages 1-4; all_findings API reference consulted on 2026-10-20.

PLATFORM

11. Risk Register

Technical explanation

Risk Register is the workspace for investigating and tracking findings. Filters and states organize treatment, but the status alone does not explain why a finding is false or why risk was accepted. Preserve the technical reasoning.

Separate triage from repair. Resolved is a treatment state; a new scan and an appropriate test provide evidence that the vulnerable path was removed. Accepted Risk is not the same as a false positive. In Progress indicates ongoing work.

Blocking and triage interact differently by category. The supplied documentation says False Positive and Won’t Fix/Nitpick can clear a failing check automatically. Accepted Risk, In Progress and Resolved do not. A formal merge exception must follow the organization’s governance process.

Use case

A genuine SQL injection is marked In Progress while a patch is prepared. The check remains governed by the blocking condition, not by the fact that someone has started work.

Before you start

Module 3; if no finding exists, record the triage exercise as not observed.

Guided lab 11

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/11
```

Expected: The last message should identify branch lab/11. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - create a new occurrence to practice triage without reusing an already-fixed finding.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: finding for triage'
```

Expected: The branch contains module 3’s SQLi.

Step 3

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 11 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 4

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 5

DryRun - open Risk Register, filter by your repo/PR and open the finding. Copy its ID into your notebook. Choose In Progress and provide a truthful reason: the test demonstrates SQL concatenation; the planned repair is parameter binding.

Expected: The decision appears in history/status. It has not fixed the code.

If it differs: If the finding is missing, check filters and run. Do not mark a real SQLi False Positive to complete the activity.

Step 6

Terminal - apply the repair, test and push. Then revisit the finding and compare the new scan to the triage record.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Repair finding under investigation'
git push
```

Expected: Tests pass and repair evidence remains distinct from the administrative decision.

Step 7

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 8

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Risk Register, pages 1-4; PR Blocking, pages 1-6.

PLATFORM

12. Code Security Intelligence

Technical explanation

Code Security Intelligence investigates code and history using natural-language questions. Documented workflows include Application Summary, Security Reviews, Architecture Risks, Incident Response, Shadow AI, Feature Ships, Resolved Risk, Vulnerability Trends, Developer Trends, New Feature Review and PR Variant Analysis.

Scope a question by organization, repository, reference and purpose. Ask for paths, lines, commits, PRs or finding IDs where available. A fluent response is not proof; distinguish observed facts, hypotheses and unavailable data.

For threat modeling, CSI helps identify entry points, assets, flows and trust boundaries. An analyst validates the proposed map and decides which hypotheses need testing. Variant investigation can find related patterns but does not prove elimination of an entire vulnerability class.

Use case

After repairing one SQLi, a team asks where other SQL operations occur and what protects them, then checks each reported path in source.

Before you start

Modules 5 and 9; Code Security Intelligence access.

Guided lab 12

Step 1

DryRun - open Code Security Intelligence and select your repo. Choose Application Summary or an equivalent available query workflow.

Expected: The scope shows the exercise repo, not every organizational repository.

If it differs: If the feature is unavailable, record the access limitation; do not attribute a generic response to CSI.

Step 2

DryRun question box - paste the following as a question, not a terminal command.

```
PROMPT:
Map HTTP entry points and SQL operations in this repository.
Cite file/line and the analyzed reference for each conclusion.
Explain how query input reaches SQL and what control applies.
Separate observed facts, hypotheses and unavailable evidence.
Do not modify code, findings or configuration.
```

Expected: The answer should provide evidence or state that it lacks it.

If it differs: If it remains generic, add app.py and ask for an answer limited to that implementation.

Step 3

Terminal - verify at least one cited line. Compare the answer to module 9’s threat model.

```
nl -ba app.py
cat .evidence/threats.txt
```

Expected: You confirm or correct one claim and record why.

If it differs: A mismatched citation may refer to a different branch/reference. Verify before trusting it.

Step 4

DryRun and notebook - save the query if supported, named “Training - SQL boundary”. Record one untested hypothesis. No PR is created.

Expected: Outcome: scoped query, checked evidence and an explicit limitation.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Code Security Intelligence, pages 1-4.

PLATFORM

13. PR Scanning Configuration

Technical explanation

Default Configuration governs repositories without a specific saved configuration. A repo assigned to a saved configuration is governed exclusively by it and belongs to only one configuration at a time. Do not treat the two as independently additive rules.

Settings separate comments, blocking, notifications, context, policies and agents. Documented defaults include enabled agents, blocking off, High threshold, comments on and notifications off. Saved changes affect the next scan rather than retroactively rewriting scan history.

Silent mode hides an agent/policy from PR comments while analysis, registry data and possible failing checks remain. General blocking is severity-based for Code Security Agents; per-agent and per-policy blocking is binary. Notification deduplication compares the highest severity with the previous scan, not finding IDs. A configuration supports up to seven policies.

Use case

A team evaluates a policy silently while collecting findings. It deliberately leaves blocking off so an invisible experimental finding does not unexpectedly prevent merges.

Before you start

Module 2; Admin permission to create an exercise-only configuration.

Guided lab 13

Step 1

DryRun - open Settings > Configuration. Create a saved configuration named Training-observe and assign only your exercise repo. Record the previous configuration before changing it.

Expected: The repo is assigned to its own saved configuration.

If it differs: Do not change the organization Default for this exercise. If Training-observe exists, check its scope before reuse.

Step 2

DryRun - in General, enable Issue Comments and Show comment for no findings; leave PR blocking off. Under Notifications, leave notifications off. Save.

Expected: You can state the saved values. The next scan will use them.

If it differs: If Save fails, do not continue assuming settings were applied.

Step 3

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/13
```

Expected: The last message should identify branch lab/13. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 4

Terminal - make a safe change to trigger the next scan.

```
printf '\nConfiguration exercise.\n' >> README.md
git add README.md
git commit -m 'Exercise scan configuration'
```

Expected: A documentation commit was created.

Step 5

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 13 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 6

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 7

DryRun - turn off Issue Comments in the training configuration and save. Terminal - push another safe change. Verify that the dashboard/check still records analysis.

```
printf '\n# Configuration observation.\n' >> app.py
git add app.py
git commit -m 'Observe scan without comments'
git push
```

Expected: Disabling comments does not disable scanning. An older comment may belong to the previous scan.

If it differs: Compare SHA and timestamp; do not treat an old comment as proof the setting was ignored.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

DryRun - re-enable comments and save. Keep Training-observe for later modules with blocking and notifications off.

Expected: The final settings are known and documented.

Step 10

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: PR Scanning Configuration, pages 1-10.

PLATFORM

14. Custom Code Policies

Technical explanation

Custom Code Policies describe organization-specific rules in natural language. An effective rule defines the required or prohibited behavior, scope, exceptions and examples. A vague “follow best practices” instruction is difficult to evaluate consistently.

Attached policies can have uniform severity, Silent mode and Blocking. Policy blocking is binary: even a Low finding can fail a check when that policy blocks. Policies do not automatically inherit Repository Context or AGENTS.md; put required information in their Background.

Test both a violating example and a compliant example. This helps distinguish rule interpretation from generic-agent findings and avoids broad rules that flag unrelated scripts or fixtures.

Use case

An organization forbids Python eval in application code. A policy makes the convention explicit while excluding documentation and test fixtures.

Before you start

Module 13; permission to create a policy.

Guided lab 14

Step 1

DryRun - open Custom Code Policies and create “Training - no eval”. Enter the following rule and Background in their respective fields.

```
RULE:
Report calls to Python eval() in application source files.
Ignore documentation and test fixtures.
Recommend explicit parsing or an allowlisted dispatch table.
BACKGROUND:
calculator.py is application code. test_* files are tests.
```

Expected: The rule has explicit behavior, scope and exception.

If it differs: RULE/BACKGROUND are instructional field labels, not API field names.

Step 2

DryRun - under Training-observe > Code Policies, attach the policy. Choose Medium, Blocking off and Silent mode off. Save.

Expected: The policy appears in the configuration governing your repo.

If it differs: Creating without attaching is insufficient. Also check the seven-policy limit.

Step 3

Terminal - enter the exercise repository and create this module's branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/14
```

Expected: The last message should identify branch lab/14. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 4

Terminal - create the violation, commit it and publish in the next step. Do not execute eval with external input.

```
cat > calculator.py <<'PYCODE'
def calculate(expression):
    return eval(expression)
PYCODE
git add calculator.py
git commit -m 'Lab: eval policy violation'
```

Expected: calculator.py contains an eval call.

Step 5

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 14 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 6

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 7

Terminal - replace it with explicit addition, check syntax and push. In DryRun distinguish policy findings from generic-agent findings.

```
cat > calculator.py <<'PYCODE'
def calculate_sum(left, right):
    return float(left) + float(right)
PYCODE
python3 -m py_compile calculator.py
git add calculator.py
git commit -m 'Replace eval with explicit operation'
git push
```

Expected: The prohibited call is gone; compilation finishes without error.

If it differs: Fix a syntax error before pushing.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Step 10

DryRun - detach the experimental policy from Training-observe and save so it does not affect later modules.

Expected: The policy may remain as a reference but is no longer active on the repo.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Custom Code Policies, pages 1-5; PR Scanning Configuration, pages 7-8.

PLATFORM

15. Repository Context

Technical explanation

Repository Context supplies security-relevant facts that may not be obvious from code, such as tenancy, shared authentication and internal library contracts. Describe verifiable controls rather than asserting that all inputs are safe.

PR analysis can use configured documents and root AGENTS.md. DeepScan can discover nested AGENTS.md; specifically configured DeepScan context replaces that source for the scan. Record which source is active before attributing different results to model behavior.

Context describes architectural invariants, while triage explains a particular finding. Policies need their own Background. AGENTS.md may also be consumed by development agents, so its statements must remain truthful and appropriate for those consumers.

Use case

An internal wrapper enforces tenant authorization before handlers. Context points to the implementation so an investigation can verify it rather than assuming that a gateway provides object-level authorization.

Before you start

Modules 2 and 13; Repository Context access.

Guided lab 15

Step 1

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/15
```

Expected: The last message should identify branch lab/15. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 2

Terminal - create a root AGENTS.md with application facts. It explicitly states that authentication does not exist.

```
cat > AGENTS.md <<'TXT'
# Training security context
app.py exposes GET /search?q= on loopback only.
There is no authentication or tenant authorization middleware.
SQLite is in memory and contains synthetic products.
SQL input must remain a bound value, not an SQL fragment.
Do not assume that a gateway validates input.
TXT
git add AGENTS.md
git commit -m 'Add verifiable repository context'
```

Expected: Every line can be checked against app.py.

If it differs: Do not invent a control simply to reduce findings.

Step 3

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 15 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 4

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 5

DryRun - open Repository Context, create "Training context" with the same facts and save. Under Training-observe > Context, select it and save again.

Expected: The document exists and is attached through the repo configuration.

If it differs: Creating a document alone may not attach it to any scan.

Step 6

Terminal - push a safe comment to test the next scan with the saved configuration.

```
printf '\n# Context exercise.\n' >> app.py
git add app.py
git commit -m 'Observe configured context'
git push
```

Expected: The next scan occurs after the context assignment.

Step 7

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 8

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Step 9

DryRun - detach the experimental context when finished or retain only facts valid for main. Record the decision.

Expected: You know which context remains active.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Repository Context, pages 1-4.

PLATFORM

16. Finding Tuning with Feedback

Technical explanation

False Positive, Won’t Fix and Accepted Risk can teach the scanner when the explanation is useful. Empty or trivial feedback supplies no reusable reasoning. For PR findings learning occurs after merge; for DeepScan it is immediate. In Progress and Resolved are not learning categories.

Learning and check clearance are separate. Accepted Risk can provide learning context but does not automatically clear a failed blocking check. False Positive and Won’t Fix/Nitpick have the documented check-clearance behavior.

Good feedback explains the verified control, where it exists and when the reasoning does not apply. Do not teach the scanner that a real lab SQLi is safe merely to complete a workflow. Use repository context for widely applicable architecture facts.

Use case

A reviewer verifies an internal authorization wrapper and supplies precise file/commit evidence explaining a genuine false positive, rather than labeling a difficult fix as false.

Before you start

Module 11. Publish only for a genuinely verified false positive; otherwise complete the local simulation.

Guided lab 16

Step 1

Terminal - prepare a feedback draft without sending anything.

```
source "$HOME/dryrun-student/session.sh"
cat > .evidence/feedback.txt <<'TXT'
Finding ID:
Proposed category:
Scanner hypothesis:
Verified control:
File, line and commit:
Why this control applies:
Limits of this explanation:
TXT
```

Expected: The file is local and does not change learning.

Step 2

Notebook - use the real SQLi as an example: ongoing work is In Progress, not FP. Explain why the defect is real.

Expected: You identify when not to send false-positive feedback.

If it differs: If you cannot prove a control, keep the investigation open.

Step 3

Optional, GitHub - only if you have another genuinely false finding, write the final comment in .evidence/feedback-final.txt using this format with the real ID and explanation.

```
COMMENT FORMAT:
@dryrunsecurity FP FINDING_ID verified technical explanation
```

Expected: The final file contains no placeholders.

If it differs: Do not send the literal example. Without a real case, skip publication.

Step 4

Optional, Terminal - verify the correct PR before publishing the reviewed file. These commands apply only to the confirmed case from the previous step.

```
read -rp 'Verified target PR number: ' TARGET_PR
gh pr view "$TARGET_PR" --web
cat .evidence/feedback-final.txt
# Run only after reviewing the real justification:
# gh pr comment "$TARGET_PR" --body-file .evidence/feedback-final.txt
```

Expected: The required exercise ends with a correct draft; publication depends on real evidence.

If it differs: Do not merge vulnerable code to trigger learning. PR learning follows merge; DeepScan learning is immediate.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Finding Tuning with Feedback, pages 1-6; PR Blocking, pages 1-6.

PLATFORM

17. PR Blocking

Technical explanation

Effective blocking requires both DryRun configuration and SCM enforcement. DryRun emits a failing check when the configured condition holds; GitHub must require that check before merge. A red check without enforcement may be merely informative.

General blocking uses a severity threshold. Per-agent and per-policy blocking is binary. For organizational rulesets, the documentation recommends the DryRun check with DryRunSecurity as source. Classic protection can expose different check names; inspect the actual run and App identity rather than guessing.

Only False Positive and Won't Fix/Nitpick automatically clear a failing check through the documented triage behavior. Accepted Risk is not an automatic merge override. GitHub plan, permissions and bypass settings influence enforcement.

Use case

A High finding appears but merge is still possible. The team discovers that main never required the check and configures an enforceable rule in the dedicated training repo.

Before you start

Module 13; exercise-repo administrator; GitHub plan supporting the rule. Do not apply this exercise to a production branch.

Guided lab 17

Step 1

DryRun - under Training-observe > General enable PR blocking with Minimum blocking risk High and save. Leave additional binary blocking off to isolate this test.

Expected: The configuration is saved only for the exercise repo.

If it differs: If the configuration is not exercise-only, return to module 13.

Step 2

Terminal - enter the exercise repository and create this module's branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/17
```

Expected: The last message should identify branch lab/17. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 3

Terminal - introduce and commit the SQLi.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m "Lab: blocking condition"
```

Expected: The change is on this branch, not main.

Step 4

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 17 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 5

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 6

Terminal - discover the actual check name and App. Copy the exact DryRun name, not another CI check.

```
SHA=$(gh pr view --json headRefOid --jq .headRefOid)
gh api "repos/$GH_REPO/commits/$SHA/check-runs" \
  --jq '.check_runs[] | {name,app:.app.slug,conclusion}'
```

Expected: You identify the current scan's context, App and conclusion.

If it differs: If the list is empty, revisit installation/execution; do not require an invented name.

Step 7

GitHub - in the exercise repo open Settings > Rules > Rulesets and create a branch ruleset named "Training-DryRun". Target main, set enforcement Active, enable Require status checks to pass and select the observed DryRun check. Choose its source App when available. Do not add student bypass. Save.

Expected: On the failing PR, GitHub indicates the required check prevents merging. Do not click Merge.

If it differs: If unavailable, ask the administrator to confirm plan/permission. Do not replace existing rules. If severity is below High, record that the threshold was not met.

Step 8

Terminal - fix and push the query; inspect the check and merge eligibility again without merging.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Remove blocking vulnerability'
git push
```

Expected: The new scan evaluates the patch. A passing check removes this blocking condition; other rules may remain pending.

Step 9

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 10

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Step 11

GitHub - disable or delete only the Training-DryRun ruleset created in this lab. DryRun - turn off Training-observe blocking and save.

Expected: Cleanup leaves pre-existing protections intact.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: PR Blocking, pages 1-6; GitHub REST Branch protection.

PLATFORM

18. Permissions

Technical explanation

DryRun distinguishes Admin and Developer roles. The supplied documentation maps GitHub administrators to Admin and members to Developer; GitLab Owner/Maintainer map to administrative access. Repository visibility also depends on SCM access.

Administrative tasks include configuration, policy and key management and initiating administrative features such as DeepScan/Insights as documented. Developers investigate authorized repositories and participate in finding treatment. A token has its own scopes; dashboard access alone does not prove API authorization.

Separate investigation, gate configuration and integration administration in the operating model. Troubleshoot missing access by checking identity, organization, App selection, role and token scope instead of replacing every credential with an administrator token.

Use case

A user can open a PR but receives a 403 from an API. The team verifies token scope and account/repo access before granting any additional privilege.

Before you start

Module 2. No user creation or token sharing is needed.

Guided lab 18

Step 1

Terminal - read your GitHub repo permissions. This is GitHub’s result, not a complete DryRun permission list.

```
source "$HOME/dryrun-student/session.sh"
gh api "repos/$GH_REPO" --jq '{full_name,permissions}'
```

Expected: Record the repo name and returned values.

Step 2

DryRun - check role and organization. Locate configuration, Access Keys and DeepScan without changing anything. Record availability.

Expected: You distinguish visible controls from actual authorization.

If it differs: Do not request another person’s password or token for comparison.

Step 3

Notebook - create a matrix with action, observed role, expected permission and evidence. If an authorized test user already exists, compare through a separate session; otherwise mark that comparison not executed.

Expected: The matrix separates SCM permissions, DryRun role and key scope.

If it differs: 401 usually points to credentials; 403 to authorization; 404 can also hide an inaccessible resource.

Step 4

Notebook - record the minimum additional access needed for a blocked lab. Do not change real permissions merely to test.

```
git status --short
```

Expected: No user or repository permissions were changed.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Permissions, pages 1-3; API Access Keys, pages 1-2.

PLATFORM

19. MCP

Technical explanation

The DryRun MCP server exposes authorized security context to an AI client. The documented endpoint is https://insights-mcp.dryrun.security/api/insights/mcp and authentication uses a bearer token. An ordinary curl GET is not a full MCP session: the client initializes the protocol and discovers tools.

Keep the roles separate: Codex is the client, MCP exposes tools, DryRun provides security context, and the local repository is the possible repair target. The key limits access. Tool names and schemas must be discovered rather than invented.

Referencing an environment variable avoids writing the literal token into the client configuration. Launch the client from the terminal where that variable is exported. Listing a configured server is not proof that a tool call succeeds.

Use case

A reviewer asks Codex to retrieve findings for one repo and cite IDs. A separate explicit request later authorizes a local patch; the first exercise is read-only.

Before you start

Module 2; Codex installed/authenticated as described in setup; complete “DryRun key” preparation before this module.

Guided lab 19

Step 1

Terminal - load the session and check Codex authentication and variable presence without printing the key.

```
source "$HOME/dryrun-student/session.sh"
codex login status
test -n "$DRYRUN_API_KEY" && echo 'DryRun key is loaded'
```

Expected: You see valid authentication and “DryRun key is loaded”.

If it differs: Without the key message, repeat DryRun key preparation in this terminal.

Step 2

Terminal - register a server named dryrun-training. The course-specific name avoids replacing an existing personal connection.

```
codex mcp add dryrun-training \
  --url https://insights-mcp.dryrun.security/api/insights/mcp \
  --bearer-token-env-var DRYRUN_API_KEY
codex mcp get dryrun-training
```

Expected: The configuration references the correct URL and variable name, not the literal token.

If it differs: If the name exists, inspect codex mcp get before changing it. Do not substitute OAuth login for bearer authentication.

Step 3

Terminal - open Codex in read-only mode. Inside it, use /mcp to inspect the connection.

```
codex --sandbox read-only
```

Expected: The connection should expose tools or show a specific error.

If it differs: Check invalid credentials, missing scopes/permissions and an environment variable not inherited by the process.

Step 4

Codex chat - ask: “Discover the DryRun tools. Query only my OWNER/REPO repository. Cite IDs and sources. Do not change code, triage or configuration.” Replace OWNER/REPO with the actual name.

Expected: A real tool call succeeds and the answer matches the repo. Zero findings differs from connection failure.

If it differs: If the agent cannot query, record the limitation instead of accepting an invented answer.

Step 5

Codex - exit with Ctrl+C when no operation is running. Back in the terminal, check local status. Keep the connection for module 24.

```
git status --short
```

Expected: No local changes were created.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: MCP, pages 1-4; Codex commands checked against the official reference.

PLATFORM

20. DryRun API

Technical explanation

The Simple API uses https://simple-api.dryrun.security/v1. Discover an account, then a repository, then a scan or finding. IDs come from actual responses; a repository UUID is not the same as OWNER/REPO, and a scan ID is not a PR number.

The repository findings endpoint lists PR findings. all_findings combines supported finding types with filters; days is required and the documented window is 1-365 days. Inspect pagination and selection defaults before presenting an export as complete.

A successful POST that queues DeepScan is not a completed scan. Check state before retrying a write whose response was lost. The supplied PDFs do not expand every configuration/triage payload schema, so use the current OpenAPI contract or the interface rather than guessing fields.

Use case

A script exports findings with their repository and execution identifiers. It preserves pagination metadata and distinguishes an HTTP error from an empty finding list.

Before you start

Module 2; DryRun key preparation. The optional POST requires permission to start DeepScan.

Guided lab 20

Step 1

Terminal - configure the base, make the first read and inspect the JSON. dryget is a course helper using curl with Authorization Bearer.

```
source "$HOME/dryrun-student/session.sh"
dryget accounts > .evidence/accounts.json
jq . .evidence/accounts.json
```

Expected: A data list shows accessible accounts.

If it differs: If curl fails, stop and read the error. Do not treat an empty file as a valid response.

Step 2

Terminal - identify the account by org_name, copy its account_id and paste it when prompted. Then list repositories and select your exercise repo’s id.

```
read -rp 'account_id: ' AID
export AID
dryget "accounts/$AID/repositories" > .evidence/repositories.json
jq . .evidence/repositories.json
read -rp 'repository id: ' RID
export RID
```

Expected: AID and RID come from responses, not PDF example IDs.

If it differs: For 404, verify account, ID and access; do not replace a UUID with a repository name.

Step 3

Terminal - export scans and findings. Inspect structure before counting.

```
dryget "accounts/$AID/repositories/$RID/scans" > .evidence/scans.json
dryget "accounts/$AID/repositories/$RID/findings" > .evidence/pr-findings.json
QUERY="days=30&repository_id=$RID&per_page=100&page=1"
dryget "accounts/$AID/all_findings?$QUERY" > .evidence/all-findings-1.json
jq . .evidence/all-findings-1.json
```

Expected: You identify results and pagination metadata.

If it differs: If another page exists, repeat with page=2 and a different output file. One page does not prove the entire history was exported.

Step 4

Optional, Terminal - only if you want a new DeepScan and have permission, run this POST once. For branch selection use module 5's interface; no unverified branch parameter is added.

```
curl --fail-with-body --silent --show-error --request POST \
  -H "Authorization: Bearer $DRYRUN_API_KEY" \
  "$DRYRUN_BASE/accounts/$AID/repositories/$RID/deepscans" \
  > .evidence/start-deepscan.json
cat .evidence/start-deepscan.json
dryget "accounts/$AID/repositories/$RID/deepscans"
```

Expected: An accepted/queued run is only a start; wait for completion in the dashboard.

If it differs: Do not repeat POST to check progress. Use GET/listing.

Step 5

Terminal - choose a completed run's ID from the list. Export results and SBOM; verify the response format before processing it with jq.

```
read -rp 'Completed DeepScan ID: ' SID
BASE="accounts/$AID/repositories/$RID/deepscans/$SID"
dryget "$BASE/results" > .evidence/deepscan-results.json
dryget "$BASE/sca_results" > .evidence/sca-results.json
dryget "$BASE/sbom" > .evidence/sbom.json
```

Expected: The files are associated with the same scan ID.

If it differs: 401: credentials; 403: authorization; 404: ID/access; 429: wait according to Retry-After; 5xx: record request_id if available.

Step 6

Notebook - save IDs, filters, window and exported page count. Do not upload tokens or sensitive responses to the course repository.

```
git status --short
```

Expected: Files under .evidence remain untracked by Git.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: DryRun API, pages 1-6; Swagger UI/OpenAPI official.

API INTEGRATIONS

21. Webhooks

Technical explanation

Webhooks deliver PR finding notifications to an endpoint. Create a destination and then enable/select it in the repository's PR configuration with the appropriate threshold. A saved destination alone does not guarantee events. Daily digests are a separate notification purpose.

The generic payload includes repository/PR metadata and a results list. Inspect an actual event before finalizing a parser. An enum containing different finding types does not prove that every scan type emits this webhook; the delivery documentation describes PR findings.

The course receiver saves JSON without sending tickets. A content hash avoids identical saved payloads, but it is not finding-level deduplication. Production integrations need durable processing, authentication compatible with the sender, retries, monitoring and finding-based identity.

Use case

A team captures a real notification before connecting Jira, preventing a parser from depending on a simplified sample that differs from its live envelope.

Before you start

Modules 2 and 13. External delivery uses three terminals: A receiver, B tunnel, C commands. cloudflared is required only for external delivery.

Guided lab 21

Step 1

Terminal A - load the session, generate a path secret and start the receiver. Copy the displayed secret for Terminal C. This process stays open; do not expect the prompt to return.

```
source "$HOME/dryrun-student/session.sh"
export WEBHOOK_PATH_KEY=$(python3 -c \
  'import secrets; print(secrets.token_urlsafe(32))')
printf '%s\n' "$WEBHOOK_PATH_KEY"
python3 "$KIT/receiver.py"
```

Expected: The receiver reports listening on 127.0.0.1:8080.

If it differs: Address already in use means port 8080 is occupied. Stop only the previous course receiver.

Step 2

Terminal C - open another terminal window, load the session and paste the same secret when prompted. Send the local sample.

```
bash
source "$HOME/dryrun-student/session.sh"
read -rsp 'Path secret from terminal A: ' WEBHOOK_PATH_KEY; echo
export WEBHOOK_PATH_KEY
curl --fail-with-body -H 'Content-Type: application/json' \
  --data-binary "@$KIT/event-example.json" \
  "http://127.0.0.1:8080/hook/$WEBHOOK_PATH_KEY"
ls .evidence/events
```

Expected: You receive accepted:true and a JSON file under .evidence/events.

If it differs: Connection refused: receiver not running. 404: wrong path/secret. 400: invalid JSON.

Step 3

Terminal B - install cloudflared using setup and start the tunnel. Record the displayed https://...trycloudflare.com URL. Leave it running.

```
cloudflared tunnel --url http://127.0.0.1:8080
```

Expected: The temporary HTTPS URL points to your local receiver.

If it differs: Do not reuse a stopped tunnel's URL. This tunnel is for testing only.

Step 4

DryRun - create a Webhooks destination with the full URL: TUNNEL_URL/hook/SECRET. Use Test if available. Under Training-observe > Notifications enable finding notifications, select this destination, choose a suitable threshold and disable deduplication for the exercise. Save.

Expected: The test reaches the receiver or produces a diagnosable delivery message.

If it differs: The test may not use the results format; do not assume it is a real finding.

Step 5

Terminal - enter the exercise repository and create this module's branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/21
```

Expected: The last message should identify branch lab/21. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 6

Terminal C - create the SQLi to generate a PR notification.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m "Lab: webhook notification"
```

Expected: The commit is on lab/21.

Step 7

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 21 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

Terminal C - list events. Enter an actual file path as EVENT_FILE and inspect repo_full_name, pr_number and results.

```
ls -lt .evidence/events
read -rp 'Event path (.evidence/events/NAME.json): ' EVENT_FILE
jq . "$EVENT_FILE"
```

Expected: You distinguish the training/dryrun-training sample from your own repo’s event.

If it differs: Without a real event, check threshold, configuration, finding and URL. No detection may mean no notification.

Step 10

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Step 11

DryRun - disable notifications and remove the temporary destination from Training-observe. Only then press Ctrl+C in terminals A and B. Keep one selected event for module 22.

Expected: No active destination points to a stopped tunnel.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Webhooks, pages 1-5; PR Scanning Configuration, pages 4-6; Cloudflare Quick Tunnels.

API INTEGRATIONS

22. Jira Integration

Technical explanation

The supplied documentation describes middleware such as Tines or Zapier rather than assuming a complete native Jira connector. A typical flow receives an event, normalizes it, filters risk, looks for an existing issue, creates or updates it and records the outcome.

This lab implements the central transformation with Python and curl as a teaching alternative. It consumes the generic results array from module 21, uses Jira Cloud REST v3 and builds an Atlassian Document Format description. The project, issue type and any extra required fields must match your Jira configuration.

The course label derives from repository plus finding ID. Searching by that label reduces duplicate tickets, but search-then-create is not atomic across concurrent workers. A production integration needs durable identity and concurrency control, not just a JQL lookup.

Use case

A repeated notification maps back to the same finding identity. The integration updates its evidence instead of opening a new issue for every scan.

Before you start

Module 21 or the kit sample. Requires Jira Cloud and a test project with Create issues permission; GitHub access does not provide Jira access.

Guided lab 22

Step 1

Browser - open your Jira project and record its key, such as LAB. Create a personal API token in your Atlassian account. This path uses an unscoped token on the site domain; if your organization requires scoped tokens/gateway access, obtain the correct base from its administrator first.

Expected: You have site, email, project key and token. This is not a DryRun token.

If it differs: Without Jira, for a local-only test run export JIRA_PROJECT=LAB and export JIRA_ISSUETYPE_ID=10001, go to step 4 and stop after generation. These are sample values; do not send the payload to a real site.

Step 2

Terminal - enter the requested values. Do not include a trailing slash in the site URL. Token input displays no characters.

```
source "$HOME/dryrun-student/session.sh"
read -rp 'Jira site (https://YOUR-SITE.atlassian.net): ' JIRA_BASE
read -rp 'Atlassian email: ' JIRA_EMAIL
read -rp 'Project key: ' JIRA_PROJECT
read -rsp 'Jira API token: ' JIRA_API_TOKEN; echo
export JIRA_BASE JIRA_EMAIL JIRA_PROJECT JIRA_API_TOKEN
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/myself" > .evidence/jira-me.json
jq '{accountId,displayName}' .evidence/jira-me.json
```

Expected: The response identifies you.

If it differs: 401: verify email/token/base. Do not proceed if the identity query fails.

Step 3

Terminal - query allowed issue types and select the desired type’s ID. Then inspect required fields. Do not assume every project uses Task or the same ID.

```
META="$JIRA_BASE/rest/api/3/issue/createmeta/$JIRA_PROJECT/issuetypes"
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$META" > .evidence/jira-types.json
jq . .evidence/jira-types.json
read -rp 'Allowed issue type ID: ' JIRA_ISSUETYPE_ID
export JIRA_ISSUETYPE_ID
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$META/$JIRA_ISSUETYPE_ID" > .evidence/jira-fields.json
jq . .evidence/jira-fields.json
```

Expected: You confirm common fields and any additional requirements.

If it differs: If extra fields are required, update the JSON with administrator guidance before POST.

Step 4

Terminal - first generate the payload from the synthetic sample. Index 0 selects the first finding in results. The script does not call Jira.

```
python3 "$KIT/jira_payload.py" "$KIT/event-example.json" 0
jq . .evidence/jira-create.json
LABEL=$(cat .evidence/jira-label.txt)
```

Expected: You see project, issuetype, summary, labels and an ADF description.

If it differs: For a real event, use module 21’s EVENT_FILE path and verify that results exists.

Step 5

Terminal - search for an issue with the same identity before creating. Stop if the search fails; failure does not mean an empty list.

```
curl --fail-with-body --silent --show-error --get \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  --data-urlencode "jql=project = \"$JIRA_PROJECT\" AND labels = \"$LABEL\"" \
  --data-urlencode 'maxResults=20' \
  "$JIRA_BASE/rest/api/3/search/jql" > .evidence/jira-search.json
jq . .evidence/jira-search.json
```

Expected: Proceed to POST only if a valid search finds no issue. If it finds one, record its key and skip creation.

Step 6

Terminal - with a reviewed payload and valid search showing no match, create the issue once. Then copy its key.

```
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-create.json \
  "$JIRA_BASE/rest/api/3/issue" > .evidence/jira-created.json
jq . .evidence/jira-created.json
read -rp 'Issue key (created or existing): ' ISSUE_KEY
```

Expected: The response contains a key, such as LAB-123. Open it in Jira and inspect its fields.

If it differs: 400: fields/ADF; 403: permission. If the response is lost, search before repeating POST to avoid a duplicate.

Step 7

Terminal - to close the test issue, query transitions and select the ID of an available completion option. IDs vary.

```
TRANS="$JIRA_BASE/rest/api/3/issue/$ISSUE_KEY/transitions"
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" "$TRANS"
read -rp 'Completion transition ID: ' TRANSITION_ID
jq -n --arg id "$TRANSITION_ID" '{transition:{id:$id}}' \
  > .evidence/jira-transition.json
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-transition.json "$TRANS"
```

Expected: Check the new status in the browser; the transition may return an empty body.

If it differs: If no completion transition exists, use the project’s authorized workflow or ask its administrator.

Step 8

Atlassian account - revoke the lab token when no consumer still needs it. Discard the terminal variable.

```
unset JIRA_API_TOKEN
```

Expected: The test issue is identified/closed and no token was stored in Git.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Jira Integration, pages 1-5; Atlassian Basic auth, Issues e Issue search REST v3.

API INTEGRATIONS

23. API Access Keys

Technical explanation

API keys authorize programmatic access independently of the interactive dashboard session. Creation requires at least one scope and the full secret is displayed once. A key may access multiple accounts according to its permissions, which is why accounts is the first discovery call.

Choose scope by consumer: read-only export, workflow automation or remediation assistance. Review usage limits in your account rather than assuming a fixed rate. Do not share a single key across unrelated consumers just because it works.

For routine rotation, create a replacement, update the consumer, verify it and revoke the old key. Revoking first can interrupt a service. During an exposure incident, immediate revocation may instead take priority according to the response process.

Use case

A developer station and a background service get separate keys, allowing the station’s key to be revoked without interrupting the service.

Before you start

DryRun key preparation; Admin permission to create/revoke keys. Use only course keys.

Guided lab 23

Step 1

DryRun - open Settings > Access Keys and identify the course key. Record its name and purpose, never the secret.

Expected: You know which consumer uses the key to be replaced.

If it differs: Do not revoke an unfamiliar shared key.

Step 2

DryRun - create “training-replacement” with the required read scopes. Copy the secret once into a secure manager. Terminal - test the replacement before changing the current key.

```
read -rsp 'Replacement DryRun key: ' DRYRUN_NEW_KEY; echo
curl --fail-with-body --silent --show-error \
  -H "Authorization: Bearer $DRYRUN_NEW_KEY" \
  'https://simple-api.dryrun.security/v1/accounts'
```

Expected: The new key reads the expected accounts.

If it differs: For 403 on other operations, review required scope rather than granting everything.

Step 3

Terminal - retain the old key only to test revocation, activate the new key and restart clients that inherited the old environment.

```
OLD_TRAINING_KEY="$DRYRUN_API_KEY"
export DRYRUN_API_KEY="$DRYRUN_NEW_KEY"
unset DRYRUN_NEW_KEY
dryget accounts
```

Expected: The read consumer works with the replacement.

Step 4

DryRun - revoke only the identified old key. Terminal - confirm it no longer authorizes and discard it.

```
curl --silent --show-error --output .evidence/revoked-response.json \
  --write-out '%{http_code}
' \
  -H "Authorization: Bearer $OLD_TRAINING_KEY" \
  'https://simple-api.dryrun.security/v1/accounts'
unset OLD_TRAINING_KEY
```

Expected: The old key is rejected; the new one still works.

If it differs: If the old key still authorizes, confirm which key was revoked before concluding.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: API Access Keys, pages 1-2.

API INTEGRATIONS

24. Remediation

Technical explanation

Remediation connects findings to AI coding tools to prepare a contextual repair. The supplied documentation includes Codex, Claude Code, Cursor, GitHub Copilot, Windsurf and VS Code. Keys, MCP and skills supply context, but the patch still needs review and verification.

The complete loop identifies the finding and SHA, reproduces the condition, prepares a minimal change, runs the regression test, reviews the diff, updates the PR and checks the new scan. An agent saying “fixed” is not a completion criterion.

For SCA, assess compatibility and regenerate dependency resolution using the project’s manager. For DeepScan, start from current code because an older finding’s line may have moved. A stable finding ID and code evidence are more useful than line position alone.

Use case

An agent restores SQL parameter binding while preserving search behavior. A test that failed before the patch passes afterward, and the final PR SHA is scanned again.

Before you start

Modules 3 and 19; key loaded; Codex authenticated. This lab installs the skill before use without depending on the next module.

Guided lab 24

Step 1

Terminal - install DryRun skills and select Codex if prompted. Choose personal (user/global) scope, outside the exercise repo, to avoid creating untracked project files before opening the branch.

```
npx skills@latest add DryRunSecurity/external-plugin-marketplace
```

Expected: The installer reports where skills were placed. Restart any already-open Codex session.

If it differs: If installation fails, record the error; do not assume the skill is available.

Step 2

Terminal - enter the exercise repository and create this module’s branch. A branch is a line of work separate from main. lab_branch is a course helper, explained in the appendix.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/24
```

Expected: The last message should identify branch lab/24. Continue only then.

If it differs: If files are modified, run git status and finish the previous work. If the branch already exists, resume with git switch and inspect its PR; do not delete it just to retry.

Step 3

Terminal - prepare the vulnerability and confirm the test fails before repair.

```
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git add app.py
git commit -m 'Lab: AI remediation target'
```

Expected: Expected injection-test failure.

Step 4

Terminal and GitHub - publish the branch and open a pull request (PR), a request to review the change before merging it. Then open the PR in your browser.

```
lab_pr "Lab 24 - DryRun training"
gh pr view --web
```

Expected: The browser shows an open PR targeting main from this module's branch. Do not click Merge.

If it differs: If the PR already exists, use gh pr view --web. If push fails, check gh auth status and git remote -v; do not expect a scan of changes that never reached GitHub.

Step 5

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 6

Terminal - open Codex. In chat, provide repo, PR number and finding ID, and request a minimal patch without automatic commit, push or merge. If there is no finding, state that the evidence is the local test, not a DryRun detection.

```
codex --sandbox workspace-write
```

Expected: Ask: "Fix SQLi in app.py using bound parameters. Preserve search. Run tests. Do not change tests, triage, configuration or severity to hide the fault."

If it differs: If the agent requests permission, review it in the client workflow. Do not approve changes outside the exercise repo.

Step 7

Terminal - after exiting the session, review the diff and run tests yourself. Only then commit and push the patch.

```
git status --short
git diff
python3 -m unittest -v
git add app.py
git commit -m 'Verified SQL injection repair'
git push
```

Expected: Both tests pass and the patch does not weaken tests or policy.

If it differs: If the agent already committed, inspect git log -1 before another commit. If tests were improperly changed, review before publishing.

Step 8

GitHub and DryRun - open Checks on the PR, then inspect the corresponding scan in DryRun. Save evidence in the terminal. Refresh while it is running. If there is no activity after roughly 10 minutes, investigate; this is a course troubleshooting checkpoint, not a product SLA.

```
lab_evidence
```

Expected: Check the SHA: it must match git rev-parse HEAD. A failed check may be the expected detection. A missing comment does not prove a clean scan.

If it differs: If no check/scan exists, verify the App installation and selected repo. If the scan finished without a finding, record that and check the diff/agents; do not conclude the vulnerability does not exist.

Step 9

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: Remediation, pages 1-2; DryRun Skill, pages 1-3.

API INTEGRATIONS

25. DryRun Skill

Technical explanation

Skills provide workflow instructions to the coding agent. The supplied documentation describes three skills covering PR authorship/review/remediation, including dryrun-finding-remediation for DeepScan and SCA findings outside active PRs. The installer detects the client; Settings > Integrations provides client-specific guidance.

The workflow should create a branch and PR, inspect security results and repeat inspection after every new commit. The finding-remediation skill uses DRYRUN_API_KEY and can address findings already in the main branch through a new repair PR.

A workflow that waits for comments must account for clean scans that do not post them. Check completed runs and current SHA instead of waiting indefinitely. Skills guide behavior; they do not grant GitHub permissions or replace branch enforcement.

Use case

A student requests a small documentation change. The agent uses a PR workflow, reports the final commit’s security result and leaves merge to the reviewer.

Before you start

Modules 19 and 24; skills installed. Use a new branch, not module 24’s closed PR.

Guided lab 25

Step 1

Terminal - load the session and check for a clean working tree. Install skills if not yet installed; otherwise no repeat is needed.

```
source "$HOME/dryrun-student/session.sh"
git switch main
git status --short
# Only if not already installed:
# npx skills@latest add DryRunSecurity/external-plugin-marketplace
codex --sandbox workspace-write
```

Expected: The client starts in the exercise repo with skills available.

If it differs: If the client does not recognize the skill, confirm installation location and restart.

Step 2

Codex chat - ask: “Use the installed DryRun skills. On branch lab/25, document in README why SQL parameters are used. Open a PR targeting main, check the final SHA’s scan and report results. Do not merge.”

Expected: The agent creates a PR and explains the result or access limitation.

If it differs: If no comment appears, ask it to inspect checks/scan; do not let it wait indefinitely for a clean-scan comment.

Step 3

Terminal - after exiting Codex, inspect the PR and compare its SHA to the checks.

```
gh pr view --json number,url,headRefOid,state
lab_evidence
git log -1 --oneline
```

Expected: The PR belongs to your repo, the diff is documentation-only and the result refers to the latest commit.

If it differs: If the PR does not exist, inspect git status and the agent's explanation before further commands.

Step 4

Optional, chat - ask dryrun-finding-remediation to list the repo's DeepScan findings with IDs without editing. Choose only real findings for future repair.

Expected: Zero findings is valid; do not add a vulnerability to main to manufacture a case.

Step 5

Terminal - close the exercise PR without merging it. This keeps main safe for the next module.

```
lab_close
git branch --show-current
git status --short
```

Expected: The current branch is main and git status --short shows no changes. Evidence remains in .evidence.

If it differs: If the PR is already closed, use git switch main only after checking for local changes. Do not use reset --hard.

Step 6

Terminal and DryRun - after finishing the course, remove only the training MCP connection, revoke keys with no remaining consumers and disable temporary integrations.

```
codex mcp remove dryrun-training
unset DRYRUN_API_KEY JIRA_API_TOKEN WEBHOOK_PATH_KEY
```

Expected: Credentials were not stored in the repo and temporary services are stopped.

If it differs: Do not remove personal connections or other consumers' keys.

Student checkpoint

In your notebook record what you did, the evidence observed and any step not executed. Explain the outcome in your own words. Proceed only when you understand differences between expected and observed results. Cleanup is included in the final applicable steps of this lab.

Source: DryRun Skill, pages 1-3; Remediation, pages 1-2.

Final exercise - demonstrate the full loop

1. Prepare a new occurrence

After completing the modules, load your session and create a new branch. Introduce SQLi, run the test and confirm it is the same failure studied in module 3. Commit and open a PR.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/final
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git add app.py
git commit -m 'Final exercise: vulnerable query'
lab_pr 'Final exercise - DryRun lifecycle'
lab_evidence
```

2. Investigate before repairing

In the PR and DryRun identify repo, SHA, scan and finding if present. Explain the input source, SQL operation and missing control. Record In Progress only for a real finding. If the scanner does not detect it, document the difference from the local proof.

3. Repair and demonstrate

Use Codex as described in module 24 or apply the deterministic repair below. Do not change the test to make it pass. Push the commit and check the final scan.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Final exercise: verified fix'
git push
lab_evidence
```

4. Submit your evidence

Include repo/PR/SHA, before/after test results, an explanation of the defect, the final scan result and limitations. If you used Jira, include the test issue key. If you lacked Jira or Codex access, mark that step not executed. This is better than presenting a simulated integration as real.

Completion means you can repeat the exercise and explain why the repair works without confusing local tests with remote evidence. Close the PR without merging and perform module 25’s cleanup.

```
lab_close
gh pr list --state open --limit 30
git status --short
```

Quick troubleshooting and recovery

Command not found

Check the tool with command -v NAME. For lab_branch, lab_pr, lab_evidence, lab_close or dryget, load session.sh. For gh/codex/jq/python3, return to installation. A new session may not have the installer’s PATH configuration; follow its Next steps.

Git and branches

Use pwd, git status and git branch --show-current before repeating commands. Resume an existing branch. Open an existing PR with gh pr view --web. Do not create another PR merely because the DryRun comment has not arrived. Understand and save local changes before switching branches.

Scan missing or no comment

Confirm App/repo, organization, PR and SHA. Check draft/ready, comments, Silent mode and scan completion. A missing comment can mean a clean scan or no execution; check/dashboard evidence distinguishes them. For a run with findings, record severity and settings before explaining blocking.

Credentials and HTTP

401 requires credential review. 403 requires authorization/scope review. 404 may mean a wrong ID or hidden resource. 429 requires respecting limits/Retry-After. For 5xx save the time, endpoint and request_id when available. Never include the token in a support request.

Asking for help

Provide module and step, command without secrets, current folder, expected result, actual message and relevant IDs. State whether the failure is local, GitHub, DryRun, Codex or Jira. Do not send screenshots exposing tokens. This lets an instructor help without repeating the entire setup.

Appendix A - complete kit source

These are teaching helpers, not official DryRun commands. The kit uses English messages so it behaves identically in both languages; the walkthrough explains outcomes. If not using the clone/ZIP, copy each block into the named file under material/laboratorios. You do not need to transcribe downloaded files. Only app.py and test_app.py go into the initial exercise repo.

lab.sh

```
# Course helpers. Source this file from the generated session.sh.
export DRYRUN_BASE='https://simple-api.dryrun.security/v1'
lab_branch() {
  test -n "$GH_REPO" || { echo 'Load session.sh first.'; return 1; }
  test -z "$(git status --porcelain)" || {
    echo 'STOP: local changes exist. Run git status and finish them.'; return 1;
  }
  git switch main && git pull --ff-only && git switch -c "$1"
}
lab_pr() {
  test "$(git branch --show-current)" != main || {
    echo 'STOP: create an exercise branch first.'; return 1;
  }
  git push -u origin HEAD || return
  gh pr create --base main --title "$1" --body \
    'DryRun student exercise. Do not merge vulnerable code.'
}
lab_evidence() {
  local n sha
  n=$(gh pr view --json number --jq .number) || return
  sha=$(gh pr view --json headRefOid --jq .headRefOid) || return
  mkdir -p .evidence
  printf 'PR %s; remote SHA %s\n' "$n" "$sha"
  git rev-parse HEAD
  gh api "repos/$GH_REPO/commits/$sha/check-runs" \
    > ".evidence/pr-$n-checks.json" || return
  gh api --paginate "repos/$GH_REPO/issues/$n/comments" \
    > ".evidence/pr-$n-comments.json" || return
  gh pr checks
  echo 'Read the check results above; failure can be an expected finding.'
}
lab_close() {
  test -z "$(git status --porcelain)" || {
    echo 'STOP: local changes exist. Finish/review them first.'; return 1;
  }
  gh pr close || return
  git switch main
}
dryget() {
  test -n "$DRYRUN_API_KEY" || {
    echo 'STOP: load DRYRUN_API_KEY in this terminal.' >&2; return 1;
  }
  curl --fail-with-body --silent --show-error \
    -H "Authorization: Bearer $DRYRUN_API_KEY" "$DRYRUN_BASE/$1"
}
```

save_session.py

```
"""Save non-secret student paths; no token is stored."""
from pathlib import Path
import re
import shlex
import sys


def save(repo, root):
    if not re.fullmatch(r'[A-Za-z0-9_.-]+/[A-Za-z0-9_.-]+', repo):
        raise ValueError('Expected OWNER/REPO, not a URL')
    root = Path(root).resolve()
    kit = root / 'material' / 'laboratorios'
    work = root / 'repo'
    if not (kit / 'lab.sh').is_file() or not (work / '.git').exists():
        raise ValueError('Expected material/laboratorios and initialized repo')
    q = shlex.quote
    content = '\n'.join([
        '# Generated course session: no credentials.',
        'export KIT=' + q(str(kit)),
        'export GH_REPO=' + q(repo),
        'cd ' + q(str(work)) + ' || return',
        'mkdir -p .evidence',
        'source "$KIT/lab.sh"',
        '',
    ])
    (root / 'session.sh').write_text(content)
    print('Session saved:', root / 'session.sh')


if __name__ == '__main__':
    if len(sys.argv) != 2:
        raise SystemExit('Usage: python3 save_session.py OWNER/REPO')
    save(sys.argv[1], Path.home() / 'dryrun-student')
```

app.py

```
"""Teaching application: loopback only; no real authentication."""
import json
import sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs, urlparse


def database():
    db = sqlite3.connect(':memory:')
    db.execute('CREATE TABLE products (name TEXT, price INTEGER)')
    db.executemany('INSERT INTO products VALUES (?, ?)',
                   [('book', 10), ('pencil', 2)])
    return db


def search(db, term):
    return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        if parsed.path != '/search':
            self.send_error(404)
            return
        term = parse_qs(parsed.query).get('q', [''])[0]
        with database() as db:
            result = search(db, term)
        body = json.dumps(result).encode()
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.end_headers()
        self.wfile.write(body)


if __name__ == '__main__':
    HTTPServer(('127.0.0.1', 8090), Handler).serve_forever()
```

test_app.py

```
import unittest
from app import database, search


class SearchTest(unittest.TestCase):
    def test_known_product(self):
        with database() as db:
            self.assertEqual(search(db, 'book'), [('book', 10)])

    def test_injection_is_literal(self):
        with database() as db:
            self.assertEqual(search(db, "' OR 1=1 --"), [])


if __name__ == '__main__':
    unittest.main()
```

change.py

```
"""Changes only the training app.py after checking its current state."""
import pathlib
import sys
p = pathlib.Path('app.py')
safe = """return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()"""
unsafe = """query = "SELECT name, price FROM products WHERE name = '" + term + "'"
    return db.execute(query).fetchall()"""
mode = sys.argv[1]
assert mode in ('unsafe', 'safe')
before, after = (safe, unsafe) if mode == 'unsafe' else (unsafe, safe)
text = p.read_text()
if text.count(before) != 1:
    raise SystemExit('Unexpected app.py state; inspect the diff.')
p.write_text(text.replace(before, after))
```

receiver.py

```
"""Teaching receiver. Saves JSON; does not forward to Jira."""
import hashlib
import hmac
import json
import os
from pathlib import Path
from http.server import BaseHTTPRequestHandler, HTTPServer

KEY = os.environ['WEBHOOK_PATH_KEY']
OUT = Path('.evidence/events')
OUT.mkdir(parents=True, exist_ok=True)


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass  # Do not log the URL containing the training path secret.

    def do_POST(self):
        if not hmac.compare_digest(self.path, '/hook/' + KEY):
            self.send_error(404)
            return
        try:
            size = int(self.headers.get('Content-Length', '0'))
            if not 0 < size <= 1024 * 1024:
                self.send_error(413)
                return
            data = json.loads(self.rfile.read(size))
            if not isinstance(data, dict):
                raise ValueError('JSON object expected')
        except (ValueError, json.JSONDecodeError):
            self.send_error(400)
            return
        raw = json.dumps(data, sort_keys=True).encode()
        digest = hashlib.sha256(raw).hexdigest()
        dest = OUT / (digest + '.json')
        try:
            with dest.open('xb') as f:
                f.write(raw)
            print('Event saved:', dest, flush=True)
        except FileExistsError:
            print('Duplicate ignored:', digest, flush=True)
        self.send_response(202)
        self.end_headers()
        self.wfile.write(b'{"accepted":true}')


if __name__ == '__main__':
    print('Listening on 127.0.0.1:8080. Stop with Ctrl+C.', flush=True)
    HTTPServer(('127.0.0.1', 8080), Handler).serve_forever()
```

jira_payload.py

```
"""Converts ONE generic webhook finding to a Jira Cloud v3 payload."""
import json
import os
import sys
from pathlib import Path

event = json.loads(Path(sys.argv[1]).read_text())
index = int(sys.argv[2]) if len(sys.argv) > 2 else 0
finding = event['results'][index]
repo = event.get('repo_full_name', 'repository-not-provided')
ident = str(finding['id'])
import hashlib
label = 'dryrun-' + hashlib.sha256(
    (repo + ':' + ident).encode()).hexdigest()[:24]
text = '\n'.join([
    'DryRun finding: ' + ident,
    'Repository: ' + repo,
    'PR: ' + str(event.get('pr_number', '')),
    'Risk: ' + str(finding.get('risk', '')),
    'File: ' + str(finding.get('filename', '')),
    'Line: ' + str(finding.get('line_start', '')),
    str(finding.get('description', '')),
    'Dashboard: ' + str(event.get('dashboard_url', '')),
])
payload = {'fields': {
    'project': {'key': os.environ['JIRA_PROJECT']},
    'issuetype': {'id': os.environ['JIRA_ISSUETYPE_ID']},
    'summary': ('[DryRun] ' + str(finding.get('type', 'Finding'))
                + ' - ' + repo)[:240],
    'labels': [label],
    'description': {'type': 'doc', 'version': 1, 'content': [
        {'type': 'paragraph', 'content': [{'type': 'text', 'text': text}]}]},
}}
Path('.evidence/jira-create.json').write_text(json.dumps(payload, indent=2))
Path('.evidence/jira-label.txt').write_text(label)
print('Payload ready. Label:', label)
```

event-example.json

```
{
  "repo_full_name": "training/dryrun-training",
  "pr_number": 1,
  "dashboard_url": "https://app.dryrun.security",
  "results": [{
    "id": "lab-sqli-001", "type": "SQL Injection",
    "filename": "app.py", "line_start": 20, "line_end": 21,
    "description": "Synthetic example: input concatenated into a query.",
    "risk": "High", "finding_type": "pullrequest"
  }]
}
```

Appendix B - sources and limitations

Basis: 25 official PDFs supplied by the participant, dated October 20, 2026. Module page references refer to those PDFs. Teaching revision and additional command checks: October 23, 2026. This student-oriented edition supersedes the previous guide.

API Access Keys - DryRun Security Docs.pdf - https://docs.dryrun.security/api-access-keys

Code Security Intelligence - DryRun Security Docs.pdf - https://docs.dryrun.security/code-security-intelligence

Custom Code Policies - DryRun Security Docs.pdf - https://docs.dryrun.security/custom-code-policies

Documentation - DryRun Security Docs.pdf - https://docs.dryrun.security/

DryRun API - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-api

DryRun Skill - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-skill

Finding Tuning with Feedback - DryRun Security Docs.pdf - https://docs.dryrun.security/finding-tuning

IaC Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/iac-scanning

Jira Integration - DryRun Security Docs.pdf - https://docs.dryrun.security/jira-integration

Language and Framework Support - DryRun Security Docs.pdf - https://docs.dryrun.security/language-support

MCP - DryRun Security Docs.pdf - https://docs.dryrun.security/mcp

PR Blocking - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-blocking

PR Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning

PR Scanning Configuration - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning-configuration

Permissions - DryRun Security Docs.pdf - https://docs.dryrun.security/permissions

Quick Start - DryRun Security Docs.pdf - https://docs.dryrun.security/quick-start

Remediation - DryRun Security Docs.pdf - https://docs.dryrun.security/remediation

Reporting - DryRun Security Docs.pdf - https://docs.dryrun.security/reporting

Repository Context - DryRun Security Docs.pdf - https://docs.dryrun.security/repository-context

Repository Scanning with DeepScan - DryRun Security Docs.pdf - https://docs.dryrun.security/deepscan

Risk Register - DryRun Security Docs.pdf - https://docs.dryrun.security/risk-register

SCA - DryRun Security Docs.pdf - https://docs.dryrun.security/sca

Secrets Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/secrets-scanning

Vulnerability Coverage Matrix - DryRun Security Docs.pdf - https://docs.dryrun.security/vulnerability-coverage-matrix

Webhooks - DryRun Security Docs.pdf - https://docs.dryrun.security/webhooks

External references

GitHub CLI - https://cli.github.com/manual/

GitHub rulesets - https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/creating-rulesets-for-a-repository

Codex CLI - https://learn.chatgpt.com/docs/codex/cli

Codex commands - https://learn.chatgpt.com/docs/cli/reference

Codex MCP - https://learn.chatgpt.com/docs/extend/mcp

Jira authentication - https://developer.atlassian.com/cloud/jira/platform/basic-auth-for-rest-apis/

Jira issues, metadata, transitions - https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issues/

Jira JQL search - https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issue-search/

Homebrew - https://brew.sh

Cloudflare Quick Tunnels - https://developers.cloudflare.com/tunnel/get-started/quick-tunnels/

DryRun OpenAPI - https://simple-api.dryrun.security/api-docs/v1/swagger.yaml

What was validated during authoring

Bash-block and Python-script syntax was checked. The safe baseline passes tests; the vulnerable version fails the injection test; the repair passes again. Session generation and Jira payload transformation were tested locally. Webhook reception was exercised with synthetic data. Live integrations require your accounts, permissions and configuration; DryRun scans, Jira issue creation and Codex execution in your accounts were not part of this validation. Publishing the materials to GitHub is separate from executing the labs.

Interfaces, plans and option names can change. Steps describe documented controls and identify administrator prerequisites. An unavailable feature must not be replaced with an invented API call.